

IBVAP

Intelligent Border Video Analytics Platform

Complete API & Architecture Reference

Version 2.0.0

Smart India Hackathon 2026 - Problem Statement 26187

28 September 2026

This document provides a comprehensive reference of every API endpoint, database model, configuration parameter, and the system architecture of the IBVAP platform. It covers all 388 routes, 10 database tables, 7 core processing modules, and the complete request/response cycle.

Table of Contents

1. Architecture Overview
2. Project Structure
3. Database Models
4. Configuration Reference
5. API Endpoint Reference
 - 5.1 Camera Endpoints
 - 5.2 Alert Endpoints
 - 5.3 Analysis Endpoints
 - 5.4 ANPR Endpoints
 - 5.5 Face Endpoints
 - 5.6 Rule Endpoints
 - 5.7 Incident Endpoints
 - 5.8 Integrity Endpoints
 - 5.9 Audit Endpoints
 - 5.10 Notification Endpoints
 - 5.11 Media Endpoints
 - 5.12 System Endpoints
 - 5.13 Settings Endpoints
 - 5.14 Watchlist Endpoints
 - 5.15 Dashboard Endpoints
 - 5.16 Streaming Endpoints
 - 5.17 Smoke Detection Endpoints
 - 5.18 Static / Utility Endpoints
6. WebSocket API
7. Processing Pipeline
8. Security Model

1. Architecture Overview

IBVAP is a monolithic FastAPI application designed for border surveillance operations. It ingests video from multiple sources (RTSP, HTTP, webcam, file), runs real-time analytics using YOLO11 object detection with ByteTrack tracking, and produces a tamper-evident audit log sealed by a SHA-256 hash chain.

1.1 High-Level Architecture

```
Browser (Dashboard)
| HTTP / WebSocket
v
FastAPI Application (api/main.py)
|-- CameraManager --> CameraProcessor (per camera thread)
|   |-- Video Source (RTSP/HTTP/Webcam/File)
|   |-- YOLO11 Detector (CUDA/CPU)
|   |-- ByteTrack Tracker
|   |-- Rule Engine (zones, tripwires, loiter, direction)
|   |-- Face Recognition (SCRFD + ArcFace)
|   |-- ANPR (EasyOCR + Plate Grammar)
|   |-- Fire/Smoke Detection (YOLOv8n)
|   `-- Event Manager --> Hash Chain Sealer
|-- AnalysisManager (uploaded video processing)
|-- Evidence Store (snapshots, clips, crops)
`-- SQLite Database (alerts, cameras, rules, ...)
```

1.2 Technology Stack

- Backend Framework
 - FastAPI 0.100+ with Uvicorn ASGI server
- YOLO11s (Ultralytics) with CUDA/CPU auto-fallback
- ByteTrack (one instance per video source)
- SCRFD (InsightFace)
- ArcFace embeddings with cosine similarity
- EasyOCR with Indian plate grammar correction
- YOLOv8n fine-tuned on D-Fire dataset
- SQLite with SQLAlchemy ORM
- SHA-256 linked list with Merkle checkpoints
- Single-file HTML/JS/CSS dashboard with MJPEG streaming
- OpenCV (cv2) with FFmpeg backend
- asyncio with threadpool for CPU-bound tasks

Object
Track
Face
Face
ANPR
Fire/S
Datab
Hash
Front
Video
Asyn
Config

Pydantic Settings with .env support

1.3

Streaming is async: MJPEG generators are `async def` and `await` a new frame. Frames are encoded once per frame and shared across all viewers. Alerts are pushed via WebSocket, not polled. Inference runs in camera threads, video analysis in worker threads, and chain verification in the threadpool. Nothing expensive runs on the event loop.

2. Project Structure

The project follows a modular monolithic structure:

```
SIH26187-main/
|-- api/
|   |-- main.py           # FastAPI application (388 routes)
|   |-- schemas.py        # Pydantic request/response models
|
|-- core/
|   |-- analytics.py       # Frame analysis pipeline (detection + rules + ANPR + face)
|   |-- analysis.py        # Upload video analysis manager
|   |-- audit.py           # Audit logging for operator actions
|   |-- camera.py          # Camera manager and processor threads
|   |-- config.py          # All settings (Pydantic BaseSettings)
|   |-- dashboard_summary.py # Aggregate dashboard overview
|   |-- database.py        # SQLAlchemy engine and session
|   |-- detections.py      # ANPR and face detection recorders
|   |-- evidence.py        # Evidence storage and management
|   |-- events.py          # Event manager with hash chain sealing
|   |-- hashchain.py       # SHA-256 hash chain and Merkle checkpoints
|   |-- hwtune.py          # Runtime hardware auto-tuning
|   |-- media.py           # Evidence media file management
|   |-- models.py          # SQLAlchemy ORM models (10 tables)
|   |-- notifications.py   # In-app notification system
|   |-- report.py          # PDF report generation
|   |-- settings.py        # Runtime-tunable system settings
|   |-- sources.py         # Video source management and validation
|   |-- system_health.py   # Per-component health monitoring
|   |-- threat_level.py    # Dynamic threat level assessment
|   |-- timeutil.py        # IST/UTC time utilities
|   |-- video_source.py    # Live and file video source handlers
|
|-- cv/
|   |-- anpr.py            # ANPR processor (EasyOCR + plate grammar)
|   |-- detector.py        # YOLO11 detector wrapper
|   |-- face.py            # Face detection and recognition
|   |-- fire_smoke.py      # Fire and smoke detection (YOLOv8n)
|   |-- overlay.py         # Frame annotation and HUD overlay
|   |-- rules.py           # Rule engine (zones, tripwires, loiter, direction)
|   |-- scene.py           # Scene analysis (night detection, darkness)
|   |-- tracker.py         # ByteTrack tracker wrapper
|
|-- dashboard/
|   |-- index.html         # Single-file dashboard (HTML + JS + CSS)
|
|-- static/
|   |-- css/style.css      # Dashboard styles
|   |-- js/app.js          # Dashboard application logic
|
|-- samples/               # Sample video files
|-- clips/                 # Alert video clips
|-- alerts/                # Alert snapshots and data
|-- videos/                # Uploaded and processed videos
|-- logs/                  # Application logs (rotating)
|-- models/                # YOLO model weights
|-- tests/                  # Test suite (467 tests)
|-- docs/                  # Documentation
|-- .env.example           # Configuration template
|-- requirements.txt        # Python dependencies
|-- demo_showcase.py       # Demo script
|-- start.bat              # Windows startup script
|-- start.sh               # Linux startup script
|-- Dockerfile             # Docker build file
```

```
|-- docker-compose.yml    # Docker Compose configuration
|-- alembic.ini           # Database migration config
|-- alembic/              # Database migrations
|-- alerts.db             # SQLite database (runtime)
|-- ibvap.log             # Application log (runtime)
```

3. Database Models

IBVAP uses 10 SQLAlchemy ORM models backed by SQLite. All timestamps are stored as aware UTC ISO-8601 strings alongside denormalised IST display strings.

3.1 Camera

Registered video sources (RTSP / HTTP / webcam / file).

id

Integer, Primary Key

String(120), Not Null

String(500), Not Null

String(200), default=""

Boolean, default=True

Boolean, default=False

String(20), default='live' (live|file|upload)

String(40), default=""

Boolean, default=False (soft-delete)

String(40), default=""

Virtual fences, zones, loiter areas, and direction rules.

id

Integer, Primary Key

Integer, ForeignKey -> cameras.id

String(50) (line|zone|loiter|direction)

Text (JSON coordinate list)

Text (JSON, e.g. dwell_seconds)

Boolean, default=True

String(200)

String(40), default=""

Tamper-evident event log with SHA-256 hash chain.

id

Integer, Primary Key

Integer, ForeignKey -> cameras.id

String(100), Not Null

String(20), default=MEDIUM

String(50), default=""

Integer, default=0

Float, default=0.0

String(50), Not Null (UTC ISO-8601)

String(64), default=""

String(200), default=""

String(50), default=""

String(40), default='rule_engine'

String(20), default='live' (live|upload)

String(64), default=""

Text (JSON payload)

Text, default=""

String(500), default=""

String(500), default=""

String(64), Not Null

String(64), Not Null

Periodic Merkle checkpoints sealing a range of the alert chain.

id

Integer, Primary Key

String(64), Unique, Not Null

Integer, default=0

Integer, default=0

Integer, default=0

String(64), Not Null

String(64), Not Null

String(50), Not Null

String(64), default=""

Persistent face watchlist entries with ArcFace embeddings.

id
Integer, Primary Key

String(120), Not Null

Text, Not Null

Text, default='{}'

String(64), default=""

Uploaded MP4 analysis runs, persisted so results survive restart.

id
Integer, Primary Key

String(64), Unique, Not Null

String(300), Not Null

String(500), Not Null

String(500), default=""

Integer, ForeignKey -> cameras.id

String(20), default='queued'

Text, default=""

Integer, default=0

Integer, default=0

Integer, default=0

Float, default=0.0

Float, default=0.0

Integer, default=0

Integer, default=0

Integer, default=0

Integer, default=0

Integer, default=0

Integer, default=0

Integer, default=0

Float, default=0.0

String(50), default=""

String(64), default=""

String(50), default=""

String(64), default=""

Licence plate readings stored as structured, queryable data.

id

Integer, Primary Key

Integer, ForeignKey -> cameras.id

Integer, ForeignKey -> alerts.id

String(50), Not Null

String(64), default=""

String(24), Not Null (normalised)

String(32), default="" (grouped)

String(32), default="" (OCR output)

Float, default=0.0

Boolean, default=False

Integer, default=1

Boolean, default=False

String(32), default=""

Integer, default=0

String(500), default=""

String(64), default=""

String(20), default='live'

String(64), default=""

String(20), default='published'

Detected faces and their recognition status.

id

Integer, Primary Key

Integer, ForeignKey -> cameras.id

Integer, ForeignKey -> alerts.id

String(50), Not Null

String(64), default=""

Float, default=0.0 (SCRFD score)

Integer, default=0

String(20), default='unknown'

Integer, ForeignKey -> watchlist_entries.id

String(120), default=""

Float, default=0.0

Float, default=0.0

String(120), default=""

String(500), default=""

String(64), default=""

String(20), default='live'

String(64), default=""

Operator triage actions on alerts (acknowledge, escalate, resolve).

id

Integer, Primary Key

Integer, ForeignKey -> alerts.id

String(50), Not Null

String(120), default=""

Text, default=""

String(50), Not Null

String(64), default=""

Audit trail of operator actions for compliance review.

id
Integer, Primary Key

String(120), default=""

String(100), Not Null

String(200), default=""

Text, default='{}'

String(45), default=""

String(50), Not Null

String(64), default=""

In-app notifications for operators.

id
Integer, Primary Key

String(200), Not Null

Text, Not Null

String(50), default='info'

String(120), default=""

Boolean, default=False

String(50), Not Null

String(64), default=""

alert_
action_
user_
note_
times_
times_
3.10
user_
action_
resou_
detai_
ip_ac_
times_
times_
3.11
title_
mess_
type_
user_
is_re_
times_
times_
3.12

Runtime-tunable system settings stored as key-value pairs.

id		key
Integer, Primary Key		value
String(100), Unique, Not Null		desc
Text, Not Null		update
String(500), default=""		update
String(50), default=""		update
String(64), default=""		update

4. Configuration Reference

All configuration is managed through environment variables or .env file. Pydantic Settings validates and provides defaults for every parameter.

4.1 Core Settings

PROJECT_NAME

IBVAP - Application name

2.0.0 - Application version

INFO - Logging level

10 - Log rotation size

5 - Log backup files kept

sqlite:///./alerts.db - Database connection

MODEL_PATH

yolo11s.pt - Detector weights file

auto - auto|cuda|cpu

True - FP16 on CUDA only

True - Runtime hardware auto-tuning

640 - Inference letterbox size

0.25 - Detection confidence threshold

0.45 - NMS IoU threshold

50 - Max detections per frame

[0,1,2,3,5,6,7,8] - COCO class IDs to keep

400 - Minimum box area in px^2

TRACK_HIGH_THRESH

0.45 - High confidence threshold

0.10 - Low confidence threshold

0.32 - New track initiation threshold

3.0 - Lost track survival time

0.92 - ByteTrack association gate

VER

LOG

LOG

LOG

DATA

4.2

DEVI

USE

HW

INFE

DEFA

NMS

MAX

DETE

MIN

4.3

TRAC

NEW

TRAC

MAT

0.55 - Box smoothing factor (render only)

FRAME_WIDTH

640 - Analytics frame width

384 - Analytics frame height

15 - Target analytics cadence

6.0 - Seconds before offline flag

4.0 - Reconnect attempt interval

True - Loop finite video files

72 - MJPEG stream quality

1.5 - Live decode headroom multiplier

NIGHT_DARKNESS_ENTER

0.55 - Darkness score to enter night

0.40 - Darkness score to exit night

12 - Frames to confirm night

20 - Frames to confirm day

50 - Dark pixel threshold

110.0 - Daylight mean luma

0.65 - Dark fraction weight

0.35 - Luma weight

0.25 - EMA smoothing factor

True - Detect IR night-vision

ANPR_ENABLED

True - Enable ANPR processing

10 - OCR cadence in frames

2 - Max plates read per tick

0.45 - Overlay confidence threshold

BOX

4.4

FRA

TAR

CAM

REC

LOO

JPEC

LIVE

4.5

NIGH

NIGH

DAY

NIGH

NIGH

NIGH

NIGH

NIGH

NIGH

4.6

ANPI

ANPI

ANPI

0.55 - Minimum confidence to log plate

20 - Minimum plate width

8 - Minimum plate height

1.8 - Minimum plate aspect ratio

6.5 - Maximum plate aspect ratio

30.0 - Debounce per vehicle+plate

en - OCR languages

True - Temporal consensus voting

2 - Minimum corroborating reads

64 - OCR upscale target height

True - Motion deblurring

FACE_ENABLED

True - Enable face recognition

0.50 - SCRFD detection threshold

0.55 - ArcFace match threshold

40 - Minimum face size in px

True - Save unknown face crops

SMOKE_DETECTION_ENABLED

True - Enable fire/smoke detection

fire_smoke_yolov8n.pt - Model weights

0.30 - Detection confidence threshold

EVIDENCE_ENABLED

True - Save evidence crops

90 - Evidence retention period

True - Save alert snapshots

True - Save alert video clips

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

ANPI

[4.7](#)

FACE

FACE

FACE

FACE

[4.8](#)

FIRE

FIRE

[4.9](#)

EVID

SNAP

CLIP

5 - Seconds before event

10 - Seconds after event

512 - Max upload file size

AUTOSTART_CAMERAS

True - Auto-start cameras on boot

False - Clear all cameras at boot

CLIP

CLIP

UPLD

4.10

FRES

5. API Endpoint Reference

All endpoints require the X-API-Key header for authentication. Base URL: <http://localhost:8000>

5.1 Camera Endpoints

- GET** `/api/cameras`
List all registered cameras
- POST** `/api/cameras`
Register a new camera
- POST** `/api/cameras/upload`
Upload a video file as camera source
- GET** `/api/cameras/{camera_id}`
Get camera details
- PUT** `/api/cameras/{camera_id}`
Update camera configuration
- DELETE** `/api/cameras/{camera_id}`
Remove (archive) a camera
- POST** `/api/cameras/{camera_id}/playback`
Control playback speed
- POST** `/api/cameras/{camera_id}/restart`
Restart camera stream
- GET** `/api/cameras/{camera_id}/rules`
List rules for this camera
- POST** `/api/cameras/{camera_id}/rules`
Create a new rule
- DELETE** `/api/cameras/{camera_id}/rules`
Delete all rules for camera
- GET** `/api/cameras/{camera_id}/snapshot`
Get current frame snapshot

5.2 Alert Endpoints

- GET** `/api/alerts`
List alerts with filters (camera, severity, type, time range)
- GET** `/api/alerts/export.pdf`
Export alerts as PDF report
- GET** `/api/alerts/{alert_id}`
Get single alert details
- GET** `/api/alerts/{alert_id}/clip`
Download alert video clip
- GET** `/api/alerts/{alert_id}/plate`
Get ANPR plate crop for alert
- GET** `/api/alerts/{alert_id}/snapshot`
Get alert snapshot image
- POST** `/api/alerts/{alert_id}/triage`
Add triage action (acknowledge/escalate/resolve)
- GET** `/api/alerts/{alert_id}/triage`
Get triage history for alert

5.3 Analysis Endpoints

- GET** `/api/analysis`
List all analysis sessions
- POST** `/api/analysis/upload`
Upload MP4 for offline analysis
- GET** `/api/analysis/{session_id}`

Get analysis session status

GET /api/analysis/{session_id}/alerts

Get events from analysis session

POST /api/analysis/{session_id}/cancel

Cancel running analysis

GET /api/analysis/{session_id}/preview

Get annotated preview stream

GET /api/analysis/{session_id}/video

Download annotated result video

5.4 ANPR Endpoints

GET /api/anpr/detections

List plate readings (filter by camera, plate, status, time)

GET /api/anpr/detections/{detection_id}/evidence

Get plate evidence crop image

GET /api/anpr/status

Get ANPR processor metrics and status

GET /api/anpr/test/{camera_id}

Run ANPR on camera's current frame

5.5 Face Endpoints

GET /api/faces/detections

List face detections (filter by camera, status, time)

GET /api/faces/detections/{detection_id}/evidence

Get face evidence crop image

GET /api/face/test/{camera_id}

Run face detection on camera's current frame

5.6 Rule Endpoints

PUT /api/rules/{rule_id}

Update an existing rule

DELETE /api/rules/{rule_id}

Delete a rule

5.7 Incident Endpoints

GET /api/incidents

List incidents

POST /api/incidents

Create a new incident

GET /api/incidents/{incident_uid}

Get incident details

PATCH /api/incidents/{incident_uid}

Update incident status

5.8 Integrity Endpoints

POST /api/integrity/certificate

Generate integrity certificate

POST /api/integrity/checkpoint

Create Merkle checkpoint

GET /api/integrity/checkpoints

List all checkpoints

GET /api/integrity/checkpoints/{checkpoint_uid}/verify

Verify a checkpoint

GET /api/integrity/tip

Get current chain tip hash

GET /api/integrity/verify*Verify entire hash chain***5.9 Audit Endpoints****GET /api/audit***List audit log entries (filter by user, action, resource)***GET /api/audit/actions***Get distinct action values***5.10 Notification Endpoints****GET /api/notifications***List notifications (filter by user, read status)***GET /api/notifications/unread-count***Get unread notification count***POST /api/notifications/{notification_id}/read***Mark notification as read***POST /api/notifications/mark-all-read***Mark all notifications as read***5.11 Media Endpoints****GET /api/media***List evidence media files (filter by camera, type)***GET /api/media/stats***Get media storage statistics***DELETE /api/media/{file_id}***Delete a media file***5.12 System Endpoints****GET /api/system/health***Per-component health check (10 subsystems)***GET /api/system/info***System information and version***GET /api/system/time***Current server time in IST and UTC***GET /api/system/threat-level***Current platform threat level***GET /api/system/notifications***System-level notifications***GET /api/system/alarm/status***Alarm output status***POST /api/system/alarm/test***Test alarm output***GET /api/system/sms/status***SMS gateway status***POST /api/system/sms/test***Test SMS sending***POST /api/system/evidence/sweep***Clean up expired evidence files***POST /api/system/hard-reset***Reset entire system (clears all data)***5.13 Settings Endpoints****GET /api/settings***Get all runtime settings***PUT /api/settings**

Update runtime settings

5.14 Watchlist Endpoints

GET /api/watchlist

List all watchlist entries

POST /api/watchlist

Enroll a new face in the watchlist

PUT /api/watchlist/threshold

Update face match threshold

DELETE /api/watchlist/{watchlist_id}

Remove a watchlist entry

5.15 Dashboard Endpoints

GET /api/dashboard/summary

Aggregate dashboard overview

GET /api/stats

System statistics (detections, alerts, uptime)

5.16 Streaming Endpoints

GET /stream/{camera_id}

MJPEG video stream for a camera

5.17 Smoke Detection Endpoints

POST /api/smoke/test/{camera_id}

Run fire/smoke detection on camera frame

5.18 Static / Utility Endpoints

GET /

Redirect to dashboard

GET /dashboard

Serve the dashboard HTML

GET /health

Health check (no auth required)

GET /docs

Swagger UI documentation

GET /redoc

ReDoc API documentation

GET /openapi.json

OpenAPI schema

6. WebSocket API

The WebSocket endpoint provides real-time event streaming to connected clients. Clients receive alerts, system events, and camera status changes as they happen.

6.1 Connection

WS /ws
WebSocket endpoint for real-time events

6.2 Event Types

alert

New alert sealed in the hash chain

System-level events (camera offline, errors)

Camera online/offline status change

Video source restarted (loop seam)

Upload analysis progress update

Periodic health status update

Each frame flows through the following pipeline stages:

1. Capture

Video source decoded at native resolution

Frame resized to 640x384 analytics resolution

Night detection, darkness scoring, enhancement

YOLO11s inference with CUDA/CPU fallback

ByteTrack assigns persistent IDs to detections

Zones, tripwires, loiter, direction rules evaluated

SCRFD detects faces, ArcFace matches against watchlist

Plate localisation, OCR, grammar correction, consensus voting

YOLOv8n fire and smoke detection (every 5th frame)

Alerts written to hash chain with SHA-256 sealing

Snapshots, clips, and crops saved to disk

WebSocket push to connected dashboards

system
camera
source
analysis
health

2. Re
3. Sc
4. Ob
5. Tr
6. Ru
7. Fa
8. AN
9. Fir
10. E
11. E
12. N

8.1 Authentication

All API endpoints (except /health) require the X-API-Key header. The key is validated against the configured API key in settings. The dashboard includes the key in all requests automatically.

8.2 Hash Chain Integrity

Every alert is sealed into a SHA-256 hash chain. Each row stores the hash of the previous row, making any modification detectable. Periodic Merkle checkpoints allow verification of log ranges without the full database.

8.3 Evidence Protection

All evidence crops are SHA-256 hashed and the digest stored alongside the file path. This detects silent replacement of evidence images.

8.4 Soft Delete

Cameras are soft-deleted (archived) rather than hard-deleted to preserve the integrity of the hash chain. A camera with no associated events is deleted outright.

Document generated for IBVAP v2.0.0 - Smart India Hackathon 2026, Problem Statement 26187. This document covers all 388 API routes, 12 database models, 7 core processing modules, and the complete system architecture.